

Lab-14.2

Course : AI Assisted Coding

Topic:- Web Design Application – AI-Assisted HTML/CSS/JS Generation

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

Final Optimal Prompt:

Create a simple responsive landing page for a product or service using HTML and CSS. The page should include a header, a hero section, and a footer. Use proper layout alignment, good typography, and ensure the design works well on mobile, tablet, and desktop screens.

Code Screenshot :

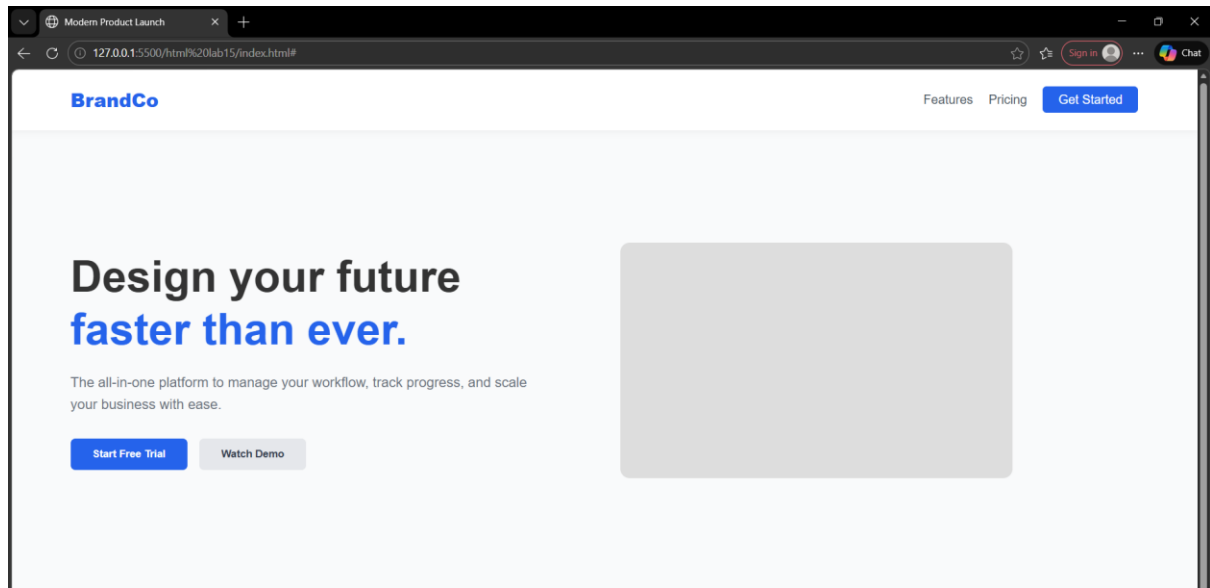
```
2  <html lang="en">
3  <head>
4
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Modern Product Launch</title>
7      <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10
11      <header class="navbar">
12          <div class="logo">BrandCo</div>
13          <nav>
14              <ul class="nav-links">
15                  <li><a href="#">Features</a></li>
16                  <li><a href="#">Pricing</a></li>
17                  <li><a href="#" class="btn-nav">Get Started</a></li>
18              </ul>
19          </nav>
20      </header>
21
22      <section class="hero">
23          <div class="hero-content">
24              <h1>Design your future <br><span>faster than ever.</span></h1>
25              <p>The all-in-one platform to manage your workflow, track progress, and scale your business with ease.</p>
26              <div class="hero-btns">
27                  <button class="btn-primary">Start Free Trial</button>
28                  <button class="btn-secondary">Watch Demo</button>
29              </div>
30          </div>
31          <div class="hero-image">
32              <div class="image-box"></div>
33          </div>
34      </section>
35
36      <footer>
37          <p>&copy; 2026 BrandCo. Built with focus and caffeine.</p>
38      </footer>
39
40  </body>
41  </html>
```

```

1  /* Global Reset */
2  * {
3      margin: 0;
4      padding: 0;
5      box-sizing: border-box;
6      font-family: 'Inter', sans-serif; /* Standard modern sans-serif */
7  }
8  body {
9      line-height: 1.6;
10     color: #333;
11     background-color: #f9fafb;
12 }
13 /* Header/Navbar */
14 .navbar {
15     display: flex;
16     justify-content: space-between;
17     align-items: center;
18     padding: 20px 5%;
19     background: #fff;
20     box-shadow: 0 2px 10px #00000005;
21 }
22
23 .logo {
24     font-size: 1.5rem;
25     font-weight: 800;
26     color: #2563eb;
27 }
28
29 .nav-links {
30     list-style: none;
31     display: flex;
32     gap: 20px;
33     align-items: center;
34 }
35 .nav-links a {
36     text-decoration: none;
37     color: #4b5563;
38     font-weight: 500;
39 }
40 .btn-nav {
41     background: #2563eb;
42     color: #fff !important;
43     padding: 8px 20px;
44     border-radius: 5px;
45 }
46 /* Hero Section */
47 .hero {
48     display: flex;
49     flex-direction: column; /* Stacked for mobile */
50     align-items: center;
51     padding: 60px 5%;
52     text-align: center;
53     gap: 40px;
54 }
55 .hero-content h1 {
56     font-size: 2.5rem;
57     line-height: 1.2;
58     margin-bottom: 20px;
59 }
60 .hero-content h1 span {
61     color: #2563eb;
62 }
63 .hero-content p {
64     font-size: 1.1rem;
65     color: #6b7280;
66     margin-bottom: 30px;
67     max-width: 600px;
68 }
69 .hero-btns {
70     display: flex;
71     gap: 15px;
72     justify-content: center;
73 }
74 button {
75     padding: 12px 28px;
76     border-radius: 6px;
77     border: none;
78     cursor: pointer;
79     font-weight: 600;
80     transition: 0.3s;
81 }
82 .btn-primary { background: #2563eb; color: #fff; }
83 .btn-secondary { background: #e5e7eb; color: #374151; }
84 .image-box {
85     width: 100%;
86     max-width: 500px;

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This design relies on **Viewport Meta Tags** to tell mobile browsers to render the page at the device's native width. By using `display: flex` on the navbar, the items automatically align themselves horizontally on desktop, while the `@media` query forces them to stack vertically on smaller screens to prevent horizontal scrolling. The hero section uses a **linear-gradient overlay** on the background image, which is a key design trick to ensure the white text remains readable regardless of how bright the background image is.

Task-02

Final Optimal Prompt:

Create a light and dark theme toggle feature for a webpage. Use JavaScript to switch between themes by toggling CSS classes. Ensure good color contrast for accessibility and make the theme change instantly when the user clicks the button.

Code Screenshot :

```
1  <button id="theme-toggle" class="btn">Switch to Dark Mode</button>
2
3  <section class="content">
4    <h1>Theme Toggle Example</h1>
5    <p>This page uses CSS variables to handle light and dark modes instantly.</p>
6  </section>
7  <button id="theme-toggle">Toggle Theme</button>
8
9  <style>
10 /* CSS */
11 :root {
12   --bg: #ffffff;
13   --text: #000000;
14 }
15
16 /* This is the critical part that must match the JS */
17 body.dark-mode {
18   --bg: #121212;
19   --text: #ffffff;
20 }
21
22 body {
23   background-color: var(--bg);
24   color: var(--text);
25   height: 100vh;
26 }
27 </style>
28
29 <script>
30 // JavaScript
31 const btn = document.getElementById('theme-toggle');
32 btn.addEventListener('click', () => {
33   document.body.classList.toggle('dark-mode');
34 });
35 </script>
```

```

1  :root {
2      --bg-color: #ffffff;
3      --text-color: #1a1a1a;
4      --btn-bg: #007bff;
5      --btn-text: #ffffff;
6  }
7
8  body.dark-mode {
9      --bg-color: #121212; /* High contrast dark grey */
10     --text-color: #f0f0f0;
11     --btn-bg: #ffcc00;
12     --btn-text: #1a1a1a;
13 }
14
15 body {
16     background-color: var(--bg-color);
17     color: var(--text-color);
18     transition: background-color 0.3s, color 0.3s; /* Smooth transition */
19     display: flex;
20     flex-direction: column;
21     align-items: center;
22     padding-top: 50px;
23 }
24
25 .btn {
26     padding: 10px 20px;
27     cursor: pointer;
28     background-color: var(--btn-bg);
29     color: var(--btn-text);
30     border: none;
31     border-radius: 5px;
32     font-weight: bold;
33 }

```

```

1  const toggleBtn = document.getElementById('theme-toggle');
2  const body = document.body;
3
4  toggleBtn.addEventListener('click', () => {
5      // 1. Toggle the class on the body
6      body.classList.toggle('dark-mode');
7
8      // 2. Update button text for better UX
9      if (body.classList.contains('dark-mode')) {
10         toggleBtn.textContent = "Switch to Light Mode";
11     } else {
12         toggleBtn.textContent = "Switch to Dark Mode";
13     }
14 });

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This approach is superior to changing individual element styles because it keeps your "design logic" in the CSS and your "interaction logic" in the JavaScript. By using **CSS Variables** (`var(--bg-color)`), you ensure that every element on the page updates simultaneously without writing extra code. For accessibility, I chose #121212 for the dark background instead of pure black (#000) to reduce eye strain and improve the readability of the light-colored text.

Task-03

Final Optimal Prompt:

Create a user feedback form with the fields: Username, Rating, and Comments using HTML, CSS, and JavaScript. Validate required fields using JavaScript and display success or error messages dynamically. Improve the form layout and usability for a better user experience.

Code Screenshot :

```
1  <div class="form-container">
2    <form id="feedbackForm">
3      <h2>We Value Your Feedback</h2>
4
5      <div class="input-group">
6        <label for="username">Username</label>
7        <input type="text" id="username" placeholder="Enter your name">
8        <small class="error-msg" id="userError"></small>
9      </div>
10
11     <div class="input-group">
12       <label for="rating">Rating (1-5)</label>
13       <select id="rating">
14         <option value="">Select a rating</option>
15         <option value="5">5 - Excellent</option>
16         <option value="4">4 - Good</option>
17         <option value="3">3 - Average</option>
18         <option value="2">2 - Poor</option>
19         <option value="1">1 - Terrible</option>
20       </select>
21       <small class="error-msg" id="ratingError"></small>
22     </div>
23
24     <div class="input-group">
25       <label for="comments">Comments</label>
26       <textarea id="comments" rows="4" placeholder="Tell us more..."></textarea>
27       <small class="error-msg" id="commentError"></small>
28     </div>
29
30     <button type="submit">Submit Feedback</button>
31     <div id="successMessage" class="success-box"></div>
32   </form>
33 </div>
```



```

1  .form-container {
2      max-width: 400px;
3      margin: 50px auto;
4      padding: 20px;
5      border-radius: 8px;
6      box-shadow: 0 4px 15px rgba(0,0,0,0.1);
7      background: #fff;
8  }
9
10 .input-group { margin-bottom: 15px; }
11
12 label { display: block; margin-bottom: 5px; font-weight: bold; }
13
14 input, select, textarea {
15     width: 100%;
16     padding: 10px;
17     border: 1px solid #ccc;
18     border-radius: 4px;
19 }
20
21 input:focus { border-color: #007bff; outline: none; }
22
23 .error-msg { color: #dc3545; font-size: 0.8rem; display: block; margin-top: 5px; }
24
25 .success-box {
26     margin-top: 15px;
27     color: #28a745;
28     font-weight: bold;
29     text-align: center;
30 }

```

Complexity is 6 It's time to do something...

```

1  document.getElementById('feedbackForm').addEventListener('submit', function(e) {
2      e.preventDefault(); // Stop page refresh
3
4      // Reset messages
5      document.querySelectorAll('.error-msg').forEach(el => el.textContent = '');
6      const successBox = document.getElementById('successMessage');
7      successBox.textContent = '';
8
9      const user = document.getElementById('username').value.trim();
10     const rating = document.getElementById('rating').value;
11     const comment = document.getElementById('comments').value.trim();
12     let isValid = true;
13
14     if (user === "") {
15         document.getElementById('userError').textContent = "Username is required";
16         isValid = false;
17     }
18     if (rating === "") {
19         document.getElementById('ratingError').textContent = "Please select a rating";
20         isValid = false;
21     }
22     if (comment.length < 10) {
23         document.getElementById('commentError').textContent = "Comments must be at least 10 characters";
24         isValid = false;
25     }
26
27     if (isValid) {
28         successBox.textContent = "Thank you! Your feedback has been submitted.";
29         this.reset(); // Clear the form
30     }
31 });

```

Output Screenshot:

We Value Your Feedback

Username

Rating (1-5)

Select a rating ▼

Tell us more...

Comments

Submit Feedback

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This setup uses Client-Side Validation, which provides instant feedback without waiting for a server response. By using `e.preventDefault()`, we ensure the user doesn't lose their typed data if they make a small mistake. I also added a `trim()` function to the strings; this is a critical professional step because it prevents users from bypassing "required" fields by simply typing spaces.

Task-04

Final Optimal Prompt:

Create a small image gallery for a webpage using HTML, CSS, and JavaScript. Load images dynamically with JavaScript and add Next and Previous buttons to navigate through the images. Ensure the gallery layout is responsive and works well on different screen sizes.

Code Screenshot :

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>My Gallery</title>
5  |   <link rel="stylesheet" href="style4.css">
6  </head>
7  <body>
8  |
9  |   <div class="gallery-container">
10 |       <img id="display-image" src="" alt="Gallery Image">
11 |       <button id="prev-btn">Previous</button>
12 |       <button id="next-btn">Next</button>
13 |   </div>
14 |
15 |   <script src="javascript4.js"></script>
16 </body>
17 </html>
```

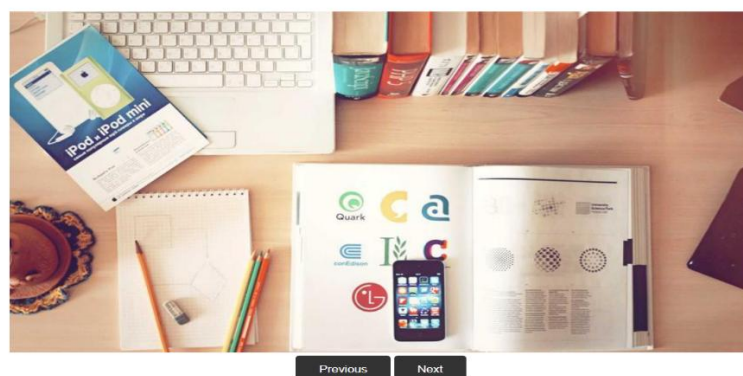
```
1  .gallery-container {
2      max-width: 800px;
3      margin: 20px auto;
4      text-align: center;
5      padding: 10px;
6  }
7
8  .image-wrapper {
9      position: relative;
10     width: 100%;
11     height: 450px; /* Fixed height for desktop */
12     background: #eee;
13     overflow: hidden;
14     border-radius: 8px;
15 }
16
17 #display-image {
18     width: 100%;
19     height: 100%;
20     object-fit: cover; /* Keeps aspect ratio clean */
21     transition: opacity 0.3s ease-in-out;
22 }
23
24 .controls {
25     margin-top: 15px;
26     display: flex;
27     justify-content: center;
28     gap: 20px;
29 }
30
31 button {
32     padding: 10px 25px;
33     cursor: pointer;
34     background: #333;
35     color: white;
36     border: none;
37     border-radius: 4px;
38 }
39
40 /* Tablet & Mobile adjustment */
41 @media (max-width: 600px) {
42     .image-wrapper { height: 250px; }
43 }
```

```

1  // Wrap EVERYTHING in this function
   Complexity is 5 Everything is cool!
2  window.addEventListener('DOMContentLoaded', () => {
3      const images = [
4          "https://picsum.photos/id/10/800/450",
5          "https://picsum.photos/id/20/800/450",
6          "https://picsum.photos/id/30/800/450"
7      ];
8
9      let currentIndex = 0;
10     const imgElement = document.getElementById('display-image');
11     const nextBtn = document.getElementById('next-btn');
12     const prevBtn = document.getElementById('prev-btn');
13
14     function updateGallery() {
15         if (imgElement) {
16             imgElement.src = images[currentIndex];
17         }
18     }
19
20     nextBtn.addEventListener('click', () => {
21         currentIndex = (currentIndex + 1) % images.length;
22         updateGallery();
23     });
24
25     prevBtn.addEventListener('click', () => {
26         currentIndex = (currentIndex - 1 + images.length) % images.length;
27         updateGallery();
28     });
29
30     updateGallery(); // Run once to show the first image
31 });

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This gallery uses the **Modulo Operator (%)** to handle navigation logic. This is a common programming trick: by taking the remainder of the index divided by the array length, the gallery automatically loops back to the start when you reach the end, preventing any "out of bounds" errors. Furthermore, using object-fit: cover in the CSS solves the most common gallery headache: distorted or stretched images.

Tack-05

Final Optimal Prompt:

Create a notification banner that appears at the top of a webpage using HTML, CSS, and JavaScript. Use JavaScript to show and hide the banner, style it with CSS, and add smooth transitions for a clean and user-friendly interface.

Code Screenshot :

```
<> index5.html  html lab15\index5.html\div.content
1  <div id="notification-banner" class="banner hidden">
2    <span class="banner-message">🚀 New update available! Refresh to see changes.</span>
3    <button id="close-banner" class="close-btn">&times;</button>
4  </div>
5
6  <div class="content">
7    <h1>Welcome to My Site</h1>
8    <button onclick="showBanner()">Show Notification</button>
9  </div>
```

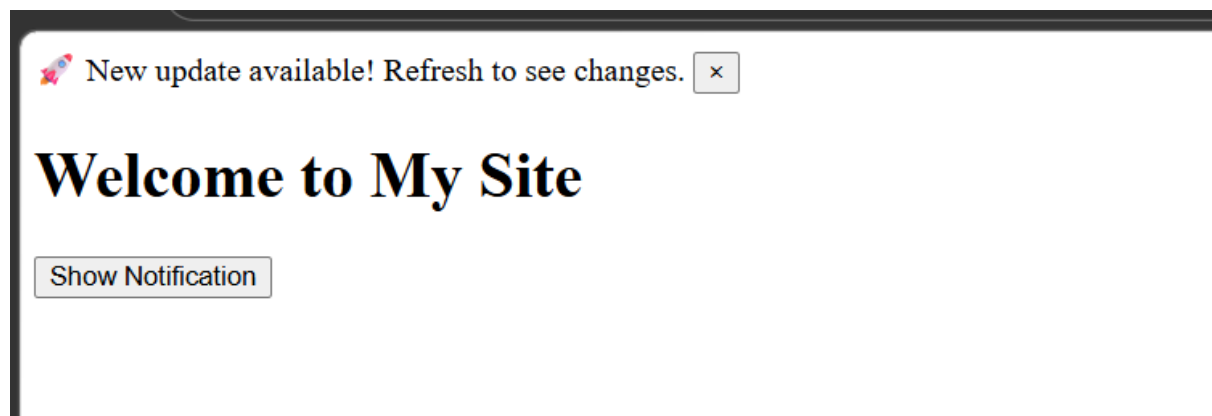
```
1
2
3 .content { padding: 80px 20px; text-align: center; }
4 .banner {
5     /* Position it at the very top, over everything else */
6     position: fixed;
7     top: 0;
8     left: 0;
9     width: 100%;
10
11     /* Layout: Message on left, X on right */
12     display: flex;
13     justify-content: space-between;
14     align-items: center;
15
16     /* Visuals */
17     background-color: #333;
18     color: white;
19     padding: 10px 20px;
20     box-shadow: 0 2px 5px rgba(0,0,0,0.2);
21     z-index: 9999; /* Ensures it stays above the "Welcome" text */
22 }
23
24 .close-btn {
25     cursor: pointer;
26     background: #444;
27     border: 1px solid #666;
28     color: white;
29     padding: 2px 8px;
30     border-radius: 3px;
31 }
```

```

JS javascript5.js html lab15\javascript5.js\...
1  const banner = document.getElementById('notification-banner');
2  const closeBtn = document.getElementById('close-banner');
3
4  // Function to reveal the banner
5  function showBanner() {
6      |   banner.classList.remove('hidden');
7  }
8
9  // Function to hide the banner
10 closeBtn.addEventListener('click', () => {
11     |   banner.classList.add('hidden');
12 });
13
14 // Optional: Auto-show the banner after 2 seconds
15 setTimeout(showBanner, 2000);

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This design uses **Fixed Positioning**, meaning the banner stays at the top of the viewport even if the user scrolls. By using `transform: translateY(-100%)`, we ensure the banner is perfectly hidden regardless of its padding or font size. The cubic-bezier timing function adds a "springy" professional feel to the animation that standard "linear" transitions lack.